

FOVA: Fast One-Shot Verifiable Aggregation for Federated Learning

Yin Zhu¹, Junqing Gong¹(✉), Kai Zhang², Shay Gueron³, Haifeng Qian¹(✉)

¹ East China Normal University, Shanghai, China.

ecnuzhuyin@163.com, jkgong@sei.ecnu.edu.cn, hfqian@cs.ecnu.edu.cn

² Shanghai University of Electric Power, Shanghai, China.

kzhang@shiep.edu.cn

³ University of Haifa, Meta

shay@math.haifa.ac.il

Abstract. In federated learning (FL), secure aggregation (SA) allows a server to compute aggregate model updates (gradients) without accessing individual client gradients. SA is intended to protect clients' local dataset from being inferred through individual gradients. However, recent NDSS 2025 work shows that even state-of-the-art SA protocols can be vulnerable, as a malicious server may reconstruct clients' datasets from aggregated gradients. This demonstrates that protecting dataset privacy requires not only gradient confidentiality but also aggregation hiding. Moreover, a malicious server may deviate from the protocol and return manipulated results, making authenticity an additional critical security goal. Supporting one-shot clients, which send a single message per iteration to reduce synchronization overhead, further increases design complexity.

To address these challenges, we propose FOVA, a fast one-shot verifiable aggregation protocol that simultaneously achieves aggregation hiding and authenticity against an actively malicious server. Notably, authenticity, defined under full participation, must be relaxed for dropout robustness due to the indistinguishability between adversarial omissions and legitimate dropout. FOVA is built upon the verifiable linearly homomorphic encryption scheme, for which we give a new construction based solely on the Paillier cryptosystem. This design enables high efficiency and allows FOVA to be integrated into existing Paillier-based FL frameworks with minimal modifications. We implement FOVA on top of an industrial FL framework. Experimental results show that, compared to the most relevant prior protocols, FOVA achieves up to three orders-of-magnitude speedup while providing stronger security guarantees.

Keywords: Federated Learning · Secure Aggregation · Verifiable Computation · Homomorphic Encryption

1 Introduction

Training machine learning (ML) models often requires combining data from multiple organizations. In many practical domains, such as banking, insurance,

and healthcare, however, data sharing is prohibited due to privacy concerns and regulatory constraints. Federated learning (FL) [15,22] addresses this challenge by enabling multiple parties to collaboratively train a global model without revealing their local datasets. In a typical FL workflow, a central server distributes the global model to clients, each client performs local training and uploads the resulting model updates (i.e., gradients). The server aggregates these updates and sends the aggregated result back to clients, who then update their local models accordingly over multiple training rounds. Despite keeping raw data local, FL remains vulnerable to privacy leakage. In particular, gradients submitted by clients may reveal sensitive information about their training data when a malicious server performs gradient inversion attacks [23,24]. To mitigate this risk, a large body of work has proposed secure aggregation (SA) protocols for FL. These protocols ensure that the server learns only the aggregated result while the individual client gradients remain confidential.

Challenges. However, privacy leakage may still occur when the aggregator behaves maliciously. In particular, the server can perform model inversion attacks (MIA) [27,36] to reconstruct clients’ private data. Recent work at NDSS 2025 [29] further demonstrates that a malicious server can distribute adversarial global models and recover local datasets from aggregated gradients, even when state-of-the-art SA protocols are deployed. Such attacks break the primary goal of SA, indicating that resistance against MIA should be treated as a fundamental requirement. Beyond privacy threats, a malicious server may also deviate from the protocol by omitting client contributions to save computational resources, or even combining such behavior with forgery to infer individual data [21]. In addition, many existing SA protocols rely on online client synchronization, which becomes problematic in practical FL deployments with large numbers of intermittently connected clients. In such settings, synchronization requirements and client dropouts can introduce significant latency and communication overhead.

Current solutions. To address these challenges, a rich line of work has been proposed. To eliminate client synchronization, two recent works at CRYPTO 2025 [13,3] introduce SA protocols with *one-shot clients*, where each client sends a single message to the server per iteration and does not need to remain online afterward. However, the follow-up work TACITA [21] identifies an attack against OPA and Willow, showing that a malicious server can infer a target client’s private data by omitting other clients’ inputs and forging carefully crafted intermediate ciphertexts. To address this issue, TACITA proposes an SA protocol with *input soundness*, ensuring that the aggregation result must include inputs from at least a threshold number of clients¹.

Nevertheless, these protocols do not prevent MIA because the aggregated gradients are still revealed to the server. A natural countermeasure is therefore to hide the aggregation result from the server, namely *aggregation hiding*. Differential privacy (DP) [12,30] has been widely studied along this direction. However, recent

¹ Under full client participation, the input soundness threshold can be set to 0, ensuring that any omission is detectable. In contrast, in settings with unstable client connectivity, redundancy is required to account for legitimate client dropout.

Table 1: Comparison of secure aggregation protocols for FL within the central server model. Only representative works are shown. MIA represents model inference attack, particularly Scale-MIA[29].

Protocol	One-shot clients	Aggregation authenticity		MIA resistance	Paillier-only construction
		Unforgeability	Soundness		
Willow [3]	●	○	○	○	—
OPA [13]	●	○	○	○	—
mPVAS [26]	○	●	●	○	—
TACITA [21]	●	○	●	○	—
Zhang et al. [35]	●	●	○	●	○
PriVeriFL [31]	●	●	○	●	○
BatchCrypt [33]	●	○	○	●	●
FOVA	●	●	●	●	●

results [29] show that a malicious server can still reconstruct training samples even when both DP and SA are applied. Another line of research adopts additively homomorphic encryption (HE), particularly the Paillier cryptosystem [25]. In these schemes, gradients are encrypted and aggregated directly in the ciphertext domain, preventing the server from learning the aggregated result. Meanwhile, each client sends only a single ciphertext per round, naturally preserving the one-shot property. However, although such designs prevent MIA, a malicious server can still omit client contributions or forge the aggregation result given the public key. To the best of our knowledge, only a few Paillier-based SA protocols [35,31,34] consider security against a malicious server. These works provide *unforgeability*, allowing clients to detect forged aggregation results. However, they do not achieve input soundness and typically rely on expensive cryptographic primitives such as digital signatures or bilinear pairings, leading to substantial overhead and limiting their practicality in real-world FL systems.

Drawbacks of current solutions. In summary, existing SA protocols for FL provide limited robustness against a malicious server. Many schemes reveal the aggregated result to the server and are therefore vulnerable to MIA. Although Paillier-based designs prevent MIA, they typically fail to simultaneously defend against omission and forgery attacks. Moreover, their reliance on heavy cryptographic primitives incurs substantial computational overhead.

1.1 Contributions

In this work, we present FOVA (**F**ast **O**ne-shot **V**erifiable **A**ggregation), a secure aggregation protocol for FL within the central server model that simultaneously satisfies the following desirable properties (Table 1):

- **One-shot clients:** Each client sends a single message directly to the server in each aggregation round and does not need to stay online afterwards. This property makes FOVA well-suited for FL scenarios involving a large number of clients with intermittent connectivity or limited availability.

- **Aggregation hiding:** The aggregated result remains hidden from the server, enabling FOVA to prevent model inference attacks. Meanwhile, residual leakage attacks [11] in FL also exploit the vulnerability of exposing aggregation results to the malicious server, making aggregation hiding an effective defense against such threats.
- **Aggregation authenticity:** FOVA simultaneously achieves unforgeability and input soundness, which together establish aggregation authenticity. This property protects against forgery and arbitrary omission by a malicious server. We note that strict input soundness, whereby clients can detect any omission of their individual contributions, holds under full client participation. To support dropout robustness under unstable participation, this guarantee is relaxed due to the inherent indistinguishability between adversarial omission and legitimate client dropout. Consequently, under arbitrary client dropout, FOVA ensures detection only for omissions exceeding a specified threshold, consistent with prior work such as TACITA [21]. A more detailed discussion is provided in Section 4.2.

Besides the main properties above, FOVA enjoys two further desirable properties:

- **Paillier-only construction:** FOVA is built upon verifiable linearly homomorphic encryption (VLHE), for which we present a new construction based solely on the Paillier cryptosystem². The design leverages Paillier’s algebraic structure to achieve authenticity without introducing additional cryptographic primitives. As Paillier-based SA has already been deployed in industrial FL frameworks such as WeBank FATE [20] via BatchCrypt [33], this design allows FOVA to be integrated into existing FL systems with minimal modifications. Most importantly, FOVA is fully compatible with BatchCrypt, and their combination enables the widely deployed Paillier-based SA protocol to simultaneously achieve efficiency and strong security with minimal changes.
- **Efficiency:** We implement FOVA³ based on BatchCrypt and evaluate it in two representative FL settings: cross-device and cross-silo. Our results show that, relative to BatchCrypt, FOVA achieves aggregate authenticity with a 6–15% increase in client-side computation, while reducing server runtime by 21–42%. Furthermore, compared with the closest prior protocols, FOVA attains up to three orders-of-magnitude speedup while additionally providing input soundness.
- **Existing defense against poisoning attack:** Poisoning attack is another concern in FL, where malicious clients may attempt to bias the aggregated result by uploading malformed gradients. A typical defense is to bound the L_2 or L_∞ norm of client input vectors [7,28]. In the context of SA, this is achieved through *input validation* on encrypted inputs, typically using zero-knowledge proofs (ZK) [2,26]. FOVA directly benefits from existing optimized ZK protocols for Paillier [8,18,19], since its ciphertexts follow the standard

² For brevity, we refer to this cryptosystem simply as “Paillier” in the remainder of the paper.

³ FOVA is open-sourced and can be found at <https://github.com/Ayin2333/FOVA>

Paillier structure. In contrast, other Paillier-based SA schemes that ensure aggregation unforgeability are incompatible with these ZK techniques. As FOVA leverages standard Paillier ZK protocols without introducing new ones, we view this as a design advantage rather than a primary contribution. We describe how FOVA can be enhanced with input validation in Appendix A. In the following, we give a high-level overview of FOVA’s underlying strategy.

1.2 Overview

In general, our goal is to construct a protocol that operates as follows: (1) each client encrypts its gradient and sends the ciphertext to the server; (2) the server aggregates the encrypted gradients using additive homomorphism and distributes the result to all clients; and (3) each client decrypts the aggregated gradient and verifies whether the result is honestly computed. If so, the client updates its model using the aggregated gradient; otherwise, it aborts. This functionality can, in principle, be realized by verifiable homomorphic encryption (VHE) schemes [1,5]. However, existing VHE schemes are too computationally expensive for practical FL scenarios and often provide capabilities beyond linear homomorphism, which are redundancy for FL workloads. Prior works [35,31,34] combine Paillier with signatures or bilinear pairings to provide partial verifiability, and can be viewed as instances of verifiable linearly homomorphic encryption (VLHE) lacking omission verification. These approaches also incur substantial computational overhead and do not provide robustness against a malicious server that selectively omits client contributions. Therefore, our goal is indeed to design an efficient and robust VLHE scheme, for which we give a formal definition in Section 3. Given the widespread deployment of Paillier in industrial FL frameworks, a Paillier-based design is especially appealing, as it can be seamlessly integrated into existing FL systems with minimal modifications. Furthermore, since Paillier already supports additive homomorphism, constructing VLHE from Paillier only requires extending it to enable verifiable homomorphic addition.

To this end, the key idea is to leverage the randomness embedded in each Paillier ciphertext. We assume familiarity with Paillier [25]. Consider three ciphertexts X_1, X_2, X_3 generated by the Paillier encryption algorithm:

$$X_i := (1 + N)^{x_i} \cdot r_i^N \bmod N^2,$$

where $N := pq$, p and q are randomly generated large primes, $x_i \in \mathbb{Z}_N$, $r_i \xleftarrow{\$} \mathbb{Z}_N^*$, and $i \in [3]$. Each plaintext x_i is encrypted using a random value r_i , which can informally be viewed as an identifier of x_i . To perform homomorphic addition, the server multiplies the ciphertexts to obtain

$$Y_{\text{true}} := \prod_{i=1}^3 X_i \bmod N^2 = (1 + N)^{x_1 + x_2 + x_3} \cdot (r_1 r_2 r_3)^N \bmod N^2.$$

An observation is that if the server operates honestly, then $r_{\text{prod}} := r_1 r_2 r_3$ is embedded in the ciphertext Y_{true} and can be regarded as the unique identifier of the value $x_1 + x_2 + x_3$. In this case, assume that r_{prod} is available and let y'

denotes the decryption of Y_{true} , one can recompute

$$Y' := (1 + N)^{y'} \cdot r_{\text{prod}}^N \bmod N^2, \quad (1)$$

and have that $Y' = Y_{\text{true}}$. The key idea is to reverse this observation in order to construct verifiable homomorphism. Suppose a homomorphic addition result Y is received from an adversarial server and decrypted to obtain y' . If one recomputes Y' using the corresponding randomness value r_{prod} (we still assume the availability of r_{prod} , whose implementation will be described later), and verifies that $Y' = Y$, it may appear that Y was honestly generated. However, this conclusion does not hold because an adversary can easily forge an incorrect yet valid Y with the knowledge of N . For example, the adversary can forge Y as

$$Y := Y_{\text{true}} \cdot (1 + N)^a \bmod N^2,$$

where $a \xleftarrow{\$} \mathbb{Z}_N$ is a non-zero value. One can check that Y still satisfies the verification condition regarding equation (1). A straightforward way to address this issue is to hide N . However, this is impractical since N^2 serves as the modulus for ciphertext multiplication, and its absence would result in a homomorphic output with an excessively large bit length. Therefore, our solution is to retain the modulus while setting it to a value that leaks N only with negligible probability. Specifically, instead of N^2 , we set the modulus to be $k \cdot N^2$, where k is a randomly generated λ -bit prime. Based on the hardness of the factoring problem, given $k \cdot N^2$, an adversary can recover N only with negligible probability $\text{negl}(\lambda)$, while keeping the bit length of the homomorphic result within a practical range. Obviously, changing the modulus from N^2 to $k \cdot N^2$ increases the bit length of the homomorphic ciphertext. A detailed analysis of the resulting additional cost is provided in Section 5. Meanwhile, hiding N from the server implies that the Paillier can no longer operate in its standard public-key setting, where N serves as the public key. However, this modification is compatible with conventional Paillier-based SA protocols. In such protocols, a Paillier key pair, including the secret key, is typically shared among all clients, thereby making Paillier actually symmetric in those protocols as well. To allow clients to obtain the identifier r_{prod} of a homomorphic ciphertext, a PRF seed is distributed with the Paillier key pair. Each client derives the random values in all ciphertexts and multiplies them to compute r_{prod} (details in Section 4).

After extending Paillier with verifiable homomorphic addition and build VLHE on it, we construct FOVA upon the VLHE. The required aggregation authenticity of FOVA reduces to the authenticity of the underlying VLHE, and its robustness against MIA reduces to the privacy of VLHE. Further, to support dropout robustness under unstable client participation, a relaxed form of authenticity is introduced. We note that, in the setting with client dropout, FOVA guarantees correctness of aggregation result, but does not enforce full participation of all online clients. See Section 4.2 for details.

1.3 Outlines

The remainder of this paper is organized as follows. In Section 2, we describe the functionality and threat model of our protocol. Section 3 presents the formal definition and our construction of VLHE. In Section 4, we present FOVA, and Section 5 provides the experimental evaluation. Finally, Section 6 concludes the paper.

2 Functionality and Threat Model

In this section, we formalize the functionality and threat model of *verifiable aggregation*, corresponding to secure aggregation against a malicious server. We consider a strong security model in which the aggregation result is guaranteed to include all clients' inputs. For simplicity, client dropout is not considered here.

Functionality. The ideal functionality of verifiable aggregation $\mathcal{F}^{\text{va}}$ is described in Fig. 1. We shall discuss five important aspects of the definition:

- *Input privacy*: Input privacy ensures that each client's individual input remains confidential. No party learns any information about a client's input except itself.
- *Aggregation hiding*: Aggregation hiding ensures that the server learns nothing about the aggregate result. The aggregate is revealed only to each client while the aggregator merely facilitates computation. Notably, aggregation hiding is essential for preventing MIA as discussed earlier.
- *Aggregation authenticity*: Aggregation authenticity ensures that clients can detect any tampering by a malicious server during the aggregation process, including the omission of client inputs or the forgery of an aggregate result.
- *Incompatibility with collusion*: Any collusion between the server and a client would trivially break aggregation hiding by enabling the server to learn the aggregated result, thereby rendering the protocol vulnerable to MIA. Consequently, robustness against client-server collusion is inherently incompatible with MIA resistance. Since resistance to MIA should be regarded as a fundamental requirement for SA, as discussed earlier, client-server collusion lies outside the security guarantees of $\mathcal{F}^{\text{va}}$. This incompatibility is not unique to SA for FL, but also appears in VHE [6] and multi-client verifiable computation [9], which can be viewed as generalizations of $\mathcal{F}^{\text{va}}$.
- *Authenticated clients*: Participation is restricted to authenticated clients, otherwise a malicious server could impersonate a client to obtain the aggregated result. Such an exploit would also trivially break aggregation hiding and render the protocol vulnerable to MIA.

Threat Model. In our protocol, there are three types of entities: a one-time dealer, clients and a central server.

- *One-time dealer*. We require a trusted dealer to set up the system, similar to the related works [26,10,35,31,33]. Though a fully trusted party is a strong assumption, we argue that it is feasible in relevant applications such as smart grids and medical data sharing, where the role can be fulfilled by a trusted

Parameters: Number of clients n , input domain $\mathbb{Z}$ and session identifier ssid .
Parties: The ideal functionality $\mathcal{F}^{\text{va}}$ interacts with n clients $P_1, \dots, P_n$, each with private input $x_i \in \mathbb{Z}$, a server S , and a simulator Sim .
Functionality:

1. Upon receiving (ssid, x_i) from client P_i , send (ssid, P_i) to notify Sim .
2. If the server S is corrupted, Sim can either choose to abort the protocol or continue.
3. If the protocol is not aborted, send $\sum_{i \in [n]} x_i$ to each client.

Fig. 1: The verifiable aggregation functionality $\mathcal{F}^{\text{va}}$.

institution or hardware manufacturer. The dealer is responsible for client authentication, as well as the generation and distribution of public/private parameters to the participating parties. After the setup, the dealer exits the protocol.

- *Clients:* The clients are assumed to be semi-honest and do not collude with the server. They follow the protocol honestly but may attempt to infer private information of other clients. In each aggregation iteration, a client can send only a single message directly to the central server over a secure channel and does not need to remain online afterward.
- *Server:* The central server is assumed to be malicious. That is, the aggregator may deviate from the protocol arbitrarily. For example, it may attempt to omit client contributions, forge aggregate results (including replay attacks), or perform model inversion attacks.

3 VLHE From Paillier Only

In this section, we give the formal definition of verifiable linearly homomorphic encryption and present a new construction. First, to clear the general structure and security requirements, we give a formal definition of VLHE.

Definition 1 (VLHE). A verifiable linearly homomorphic encryption scheme Π^+ consists of five algorithms ($\text{Gen}, \text{Enc}, \text{Dec}, \text{Eval}, \text{Verify}$) as follows:

- $(\text{pp}, \text{sk}) \leftarrow \text{Gen}(1^\lambda)$: On input the security parameter 1^λ , the key generation algorithm outputs a public parameter pp and a secret key sk .
- $X \leftarrow \text{Enc}(\text{sk}, x, r)$: On input the secret key sk , an input x and a uniformly sampled identifier r , the encryption algorithm outputs a ciphertext X .
- $x \leftarrow \text{Dec}(\text{sk}, X)$: On input the secret key sk and a ciphertext X , the decryption algorithm outputs x .
- $Y \leftarrow \text{Eval}(\text{pp}, (X_1, \dots, X_n))$: On input the public parameter pp and n ciphertexts $(X_1, \dots, X_n)$, this homomorphic addition algorithm outputs a ciphertext Y .
- $\perp/1 \leftarrow \text{Verify}(\text{sk}, y, (r_1, \dots, r_n), Y)$: On input the secret key sk , the decryption result y , n identifiers $(r_1, \dots, r_n)$ and a homomorphic ciphertext Y , the verification algorithm outputs either $\perp$ or 1, indicating verification failure or success, respectively.

We require that, a VLHE scheme $\Pi^+ := (\text{Gen}, \text{Enc}, \text{Dec}, \text{Eval}, \text{Verify})$ satisfies the following properties:

- *Correctness*: Intuitively, Π^+ is correct if any honest computation will decrypt to the expected result. Formally, given the security parameter $\lambda \in \mathbb{N}$, it holds that for any x in the domain, we have

$$\Pr[\text{Dec}(\text{sk}, \text{Enc}(\text{sk}, x, r)) = x] = 1,$$

where $r \xleftarrow{\$} \mathbb{N}$, $(\text{pp}, \text{sk}) \leftarrow \text{Gen}(1^\lambda)$.

- *Completeness*: Intuitively, Π^+ is complete if Verify will always accept an honestly computed result. Formally, given the security parameter $\lambda \in \mathbb{N}$, for any $n, x_1, \dots, x_n \in \mathbb{N}$ such that $y := \sum_{i=1}^n x_i$ is in the range, we have

$$\Pr[\text{Verify}(\text{sk}, y, (r_1, \dots, r_n), Y) \neq \perp] = 1,$$

where $(\text{pp}, \text{sk}) \leftarrow \text{Gen}(1^\lambda)$, $X_i \leftarrow \text{Enc}(\text{sk}, x_i, r_i)$, $r_i \xleftarrow{\$} \mathbb{N}$ for $i \in [n]$ and $Y \leftarrow \text{Eval}(\text{pp}, (X_1, \dots, X_n))$.

- *Privacy*: Intuitively, the privacy of Π^+ requires that the ciphertext reveals nothing about the plaintext. Formally, given the security parameter $\lambda \in \mathbb{N}$, it holds that for any PPT adversary $\mathcal{A}$, we have

$$\Pr \left[b' = b \mid \begin{array}{l} (\text{pp}, \text{sk}) \leftarrow \text{Gen}(1^\lambda); \\ x_0, x_1 \leftarrow \mathcal{A}^{\text{Enc}(\text{sk}, \cdot, \cdot)}(\text{pp}); \\ b \xleftarrow{\$} \{0, 1\}; r \xleftarrow{\$} \mathbb{N}; \\ X \leftarrow \text{Enc}(\text{sk}, x_b, r); \\ b' \leftarrow \mathcal{A}^{\text{Enc}(\text{sk}, \cdot, \cdot)}(X) \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda).$$

- *Authenticity*: Intuitively, the authenticity requires that an adversary cannot make Verify accept an incorrect answer. Formally, given the security parameter λ , Π^+ achieves authenticity if for any $n, x_1, \dots, x_n \in \mathbb{N}$ such that $\sum_{i=1}^n x_i$ is in the range, it holds that for any PPT adversary $\mathcal{A}$, the following advantage $\text{Adv}_{\mathcal{A}, \Pi}^{\text{Aut}}$ is a negligible function in λ .

$$\text{Adv}_{\mathcal{A}, \Pi}^{\text{Aut}}(\lambda) := \Pr \left[\begin{array}{l} y \neq \sum_{i=1}^n x_i \\ \cap \\ v \neq \perp \end{array} \mid \begin{array}{l} (\text{pp}, \text{sk}) \leftarrow \text{Gen}(1^\lambda); r_i \xleftarrow{\$} \mathbb{N}, i \in [n]; \\ X_i \leftarrow \text{Enc}(\text{sk}, x_i, r_i), i \in [n]; \\ Y \leftarrow \mathcal{A}^{\text{Enc}(\text{sk}, \cdot, \cdot)}(\text{pp}, (X_1, \dots, X_n)); \\ y \leftarrow \text{Dec}(\text{sk}, Y); \\ v \leftarrow \text{Verify}(\text{sk}, y, (r_1, \dots, r_n), Y) \end{array} \right]$$

As discussed in Section 1.2, given a standard Paillier scheme, supporting verifiable homomorphic addition can be achieved by modifying the modulus used in the homomorphic addition and incorporating a corresponding verification algorithm. We present the concrete construction below. Let λ denote the security parameter. Let $\text{GenPrimes}()$ be a polynomial-time algorithm that, on input λ , outputs λ -bit primes, except with negligible probability in λ . Let $\Phi(\cdot)$ denote Euler's totient function.

Construction 1 (VLHE from Paillier only) *A verifiable linearly homomorphic encryption scheme Π^+ is a tuple of algorithms (Gen, Enc, Dec, Eval, Verify), where each component is specified as follows:*

- $(\text{pp}, \text{sk}) \leftarrow \text{Gen}(1^\lambda)$: Run $\text{GenPrimes}(1^\lambda)$ to obtain (p, q, k) . Compute $N := pq, \Phi(N) = (p-1)(q-1)$ and return

$$\text{pp} := k \cdot N^2, \quad \text{sk} := (N, \Phi(N)).$$

- $X \leftarrow \text{Enc}(\text{sk}, x, r)$: Compute and return

$$X := (1 + N)^x \cdot r^N \bmod N^2.$$

- $x \leftarrow \text{Dec}(\text{sk}, X)$: Compute and return

$$x := \frac{(X^{\Phi(N)} \bmod N^2) - 1}{N} \cdot \Phi(N)^{-1} \bmod N.$$

- $Y \leftarrow \text{Eval}(\text{pp}, (X_1, \dots, X_n))$: Compute and return

$$Y := \prod_{i=1}^n X_i \bmod \text{pp}.$$

- $\perp/1 \leftarrow \text{Verify}(\text{sk}, y, (r_1, \dots, r_n), Y)$:
1. Compute

$$Y' := (1 + N)^y \cdot \left(\prod_{i=1}^n r_i \right)^N \bmod N^2.$$

- 2. If $Y' \neq (Y \bmod N^2)$, return $\perp$, else return 1.

Theorem 1. *Let Π^+ be the verifiable linearly homomorphic encryption scheme in Construction 1. Then, Π^+ satisfies correctness and completeness defined in Definition 1.*

Theorem 2. *Let Π^+ be the verifiable linearly homomorphic encryption scheme in Construction 1. Then, Π^+ satisfies privacy and authenticity defined in Definition 1.*

We present proofs for Theorem 1 and Theorem 2 in Appendix B.1 and B.2.

4 FOVA: Fast One-Shot Verifiable Aggregation

In this section, we construct FOVA, a fast one-shot verifiable aggregation protocol, whose goal is implementing $\mathcal{F}^{\text{va}}$ defined in Fig. 1. FOVA relies on VLHE (Construction 1) and PRF. As discussed in Section 2, only clients that authenticated by the dealer can participate in the protocol.

4.1 Construction of FOVA

Before presenting the detailed construction, we consider a common challenge in all Paillier-based SA protocols and provide a mitigation. This mitigation also helps reduce the overall overhead of FOVA.

Common challenge in Paillier-based SA. In FL, Paillier-based SA requires a setup phase in which a Paillier key pair must be securely synchronized among all clients, which introduces significant communication overhead. Since FOVA is also built on the Paillier cryptosystem, it inherits this challenge as well. An observation is that, since the Paillier key pair is shared by all clients, it effectively acts as a symmetric key. Therefore, instead of directly synchronizing the key pair, we synchronize a PRF seed and derive the common key pair from it using a pseudorandom function. Although this approach does not completely eliminate the need for synchronization, it mitigates the overhead, especially when frequent key updates are required. For this purpose, we modify the original prime generation algorithm `GenPrimes` used in the key generation algorithm `Gen` of Construction 1. The modified `GenPrimes` takes the security parameter 1^λ and a PRF seed s as input, and iteratively executes the following steps until a prime is found: (1) compute $\text{PRF}_s(\text{"prime"} \parallel \text{salt})$ to obtain a pseudorandom $(\lambda - 1)$ -bit integer p' , where salt is initialized to 0 before the iteration; (2) construct $p := 1 \parallel p'$; (3) apply the Miller–Rabin primality test to p with security parameter 1^λ ; (4) if p passes the test, return p ; otherwise, set $\text{salt} := \text{salt} + 1$ and repeat. To incorporate the modified `GenPrimes`, the key generation algorithm `Gen` also takes the seed s as an additional input. The details are illustrated in Fig. 2.

– $(\text{pp}, \text{sk}) \leftarrow \text{Gen}(1^\lambda, s)$: On input the security parameter 1^λ and a PRF seed s , the key pair generation algorithm runs `GenPrimes` $(1^\lambda, s)$ to obtain common λ -bit primes (p, q, k) . Compute $N := pq$, $\Phi(N) := (p - 1)(q - 1)$ and return

$$\text{pp} := k \cdot N^2, \quad \text{sk} := (N, \Phi(N)).$$

Fig. 2: The optimized key pair generation algorithm for the extended Paillier.

Detailed construction of FOVA. We now present the detailed construction of FOVA in Fig. 3. In general, FOVA operates in four phases: setup, encryption, aggregation, and verification. During the setup phase, the one-time dealer assigns each client a unique identifier and securely distributes the shared PRF seed. Using this seed, each client can deterministically derive the parameters of the verifiable linearly homomorphic encryption scheme. In the encryption phase, each client derives its randomness, encrypts its private input using this randomness, and securely sends the resulting ciphertext to the server. During the aggregation phase, the server performs homomorphic addition over all received ciphertexts and returns the aggregated ciphertext to each client. Finally, in the verification phase, each client derives all the randomness and verifies the correctness of the result returned by the server. Throughout the entire process, each client sends only one message to the server, thereby preserving the one-shot property.

Theorem 3. *The protocol FOVA in Figure 3 securely implements the functionality $\mathcal{F}^{\text{va}}$ against an active adversary corrupting the server.*

We present the proof for Theorem 3 in Appendix B.3.

Let $\text{PRF} : \{0,1\}^\lambda \times \{0,1\}^* \rightarrow \mathbb{Z}_N^*$ be a pseudorandom function, and let $\Pi^+ = (\text{Gen}, \text{Enc}, \text{Dec}, \text{Eval}, \text{Verify})$ denote the verifiable linearly homomorphic encryption scheme defined in Construction 1.

- Setup phase:
 1. The one-time dealer assigns each client a unique identifier $i \in [n]$ and securely distribute the shared PRF seed $s \in \{0,1\}^\lambda$ to each client.
 2. Each client computes

$$(\text{pp}, \text{sk}) \leftarrow \text{Gen}(1^\lambda, s).$$
 3. The parameters n and pp are published.
- **Encryption** phase for client $P_i (1 \leq i \leq n)$ in the j th iteration:
 1. After obtaining the private input x_i , P_i computes $r_{i,j} \leftarrow \text{PRF}_s(i||j)$ and

$$X_i \leftarrow \text{Enc}(\text{sk}, x_i, r_{i,j}).$$
 2. P_i securely sends X_i to the server.
- **Aggregation** phase for the server in the j th iteration: After receiving X_i from client $P_i (1 \leq i \leq n)$, the server computes

$$Y \leftarrow \text{Eval}(\text{pp}, (X_1, \dots, X_n)),$$
 and sends Y to all clients.
- **Verification** phase for client $P_i (1 \leq i \leq n)$ in the j th iteration:
 1. P_i receives Y from the server and computes

$$y \leftarrow \text{Dec}(\text{sk}, Y).$$
 2. P_i computes $r_{i',j} \leftarrow \text{PRF}_s(i'||j)$ for $i' \in [n] \setminus i$ and runs the verification algorithm

$$v \leftarrow \text{Verify}(\text{sk}, y, (r_{1,j}, \dots, r_{n,j}), Y).$$

If $v = \perp$, it aborts; otherwise, it accepts y .

Fig. 3: Construction of FOVA.

Accelerate online computation. During the encryption and verification phases, each client in FOVA uses PRF to derive common random values for verifiability. However, computing the PRF introduces a non-negligible computational overhead for clients. To mitigate this cost, a tedious but effective approach is to employ preprocessing. Since all parameters required by the PRF are known during the setup phase, its computation can be performed offline before encryption begins. The experimental results in Section 5 further confirm the necessity and effectiveness of this preprocessing strategy.

Primitive-free deployment. An advantage of FOVA is its Paillier-only construction. This property makes FOVA can be seamless integrated into existing Paillier-based SA protocols with minimal modifications. We provide a plugin⁴ that upgrades BatchCrypt to FOVA. We present a brief illustration of the required

⁴ https://github.com/Ayin2333/FOVA/tree/main/BatchCrypt_Plugin

modifications here. A conventional Paillier-based SA typically runs in four phases: setup, encryption, aggregation, and decryption. Let n represent the number of the clients.

- Setup phase: In the conventional setup phase, each client securely obtains the Paillier parameters (pk, sk) with $pk = N$, $sk = (N, \Phi(N))$. To deploy FOVA, this phase is modified as follows: instead of distributing the standard Paillier parameters, each client securely receives a random PRF seed s and deterministically derives the verifiable linearly homomorphic encryption parameters (pp, sk) from s using the PRF, where the only difference is that $pp = k \cdot N^2$.
- Encryption phase: In the conventional encryption phase, each client encrypts its private input x_i using the standard Paillier encryption algorithm, producing the ciphertext X_i :

$$X_i \leftarrow (1 + N)^x \cdot r^N \bmod N^2.$$

where $r \xleftarrow{\$} \mathbb{Z}_N^*$. Each client then sends X_i to the server. To deploy FOVA, the procedure remains unchanged except that r is no longer sampled randomly. Instead, it is deterministically generated as $r_{i,j} := \text{PRF}_s(i||j)$, where s is the shared PRF seed, i denotes the client identifier, and j is the iteration number.

- Aggregation phase: In the conventional aggregation phase, the server receives X_i from client P_i ($1 \leq i \leq n$) and computes

$$Y := \prod_{i=1}^n X_i \bmod N^2.$$

The server then sends Y to each client. In FOVA, the process remains the same, except that the modulus is set to kN^2 instead of N^2 .

- Decryption/Verification phase: In the conventional decryption phase, each client receives Y from the server and applies the standard decryption algorithm to obtain y , the sum of all clients' inputs. To deploy FOVA, this process is modified as follows: after obtaining y by decrypting Y , each client derives the random values associated with the ciphertexts of all clients, and verifies the validity of Y using these values. If the verification succeeds, the client accepts y ; otherwise, it aborts.

4.2 FOVA with Dropout Robustness

In the following, we discuss two existing strategies for achieving dropout robustness while maintaining input soundness. Both strategies are compatible with FOVA.

Contradiction. We start with an observation in the context of SA: there exists an inherent contradiction between arbitrary dropout robustness and input soundness when the aggregator is a central malicious server. If a protocol ensures input soundness, any omission of client contributions must be detected and rejected. In contrast, robustness against arbitrary client dropout requires that missing contributions do not hinder the aggregation process. Consequently, if

an SA protocol tolerates arbitrary dropout, a malicious server can exploit this flexibility by falsely claiming that certain clients have dropped out, thereby omitting their inputs without detection. This implies that full input soundness cannot be maintained under arbitrary dropout, and any practical solution must introduce a trade-off between robustness and soundness. Prior works [26,21] have proposed two kinds of such strategies, both of which are also applicable to FOVA.

Planned dropout. Following the approach in [26], clients pre-select a set of rounds for non-participation instead of dropping out arbitrarily. This strategy can be directly applied to FOVA: if a client C_i expects to be offline in round j , it can proactively submit $H(r_{i,j})$ to the server while still online. This remains secure as the hash of $r_{i,j}$ leaks no information about N (recall that FOVA requires N to be hidden from the server). To justify a dropout claim for C_i in round j , the server must present $H(r_{i,j})$ to other clients for verification, thereby ensuring dropout robustness. For soundness, consider that client C_i participates in round j and sends the ciphertext $X_i := (1 + N)^{x_i} \cdot r_{i,j}^N \pmod{N^2}$. If the server attempts to omit C_i 's contribution while producing a valid aggregated result, it must provide $H(r_{i,j})$, which requires extracting $r_{i,j}$ from X_i . This is infeasible except with negligible probability, ensuring the soundness property. However, this strategy offers limited flexibility while suitable for cross-silo FL with stable participants, it remains less applicable to cross-device FL where clients may drop out unexpectedly due to network instability or power constraints.

Limited dropout rate. In [21], clients are allowed to drop out arbitrarily, however, a threshold is imposed on the total number of dropouts. If the number of clients that the server claims to have dropped out exceeds this threshold, the aggregated ciphertext becomes undecryptable. Although a malicious server can still omit some client contributions, the threshold effectively bounds the potential impact. This strategy is also compatible with FOVA by introducing a public parameter during the setup phase, which specifies the maximum allowable rate of dropout clients. At the same time, each client sends its ciphertext together with an identifier to the server, and the server returns the aggregated result along with the identifiers of the dropout clients. The remaining clients can then remove the randomness associated with the dropout clients. This strategy sacrifices full soundness guarantees, whereby a malicious server may exploit small scale of dropout to bias aggregation results or isolate small groups of clients. However, it achieves full robustness against client dropout, making it more suitable for cross-device FL settings, where clients often operate in unstable networks and may drop out randomly.

5 Implementation and Evaluation

In this section, we evaluate the performance of FOVA. To reflect real-world deployment and to test the compatibility of FOVA with optimizations for Paillier-based SA, we implement FOVA on top of BatchCrypt. Although our implementation is based on BatchCrypt, it can be seamlessly integrated into other Paillier-based frameworks with minimal modifications. We first describe the experimental setup

in Section 5.1. Then, in Section 5.2, we evaluate the concrete cost of enhancing a Paillier-based SA to FOVA. In Section 5.3 and 5.4, we present a comparison of FOVA with related schemes in two typical FL settings.

5.1 Experimental Setup

In section 5.2, we quantify the concrete overhead introduced by the enhancement, i.e., the transition from conventional Paillier-based SA to FOVA. This quantification helps real-world users evaluate the trade-off between stronger security and higher cost. To this end, we adopt the widely used BatchCrypt framework as a representative Paillier-based SA protocol and conduct experiments on two representative applications. The first is a 3-layer fully connected neural network trained on the FMNIST dataset [32], with a batch size of 128 and the Adam optimizer. The second is AlexNet [17] trained on the CIFAR-10 dataset [16], also with a batch size of 128, using the RMSprop optimizer with a decay rate of 10^{-6} .

In Sections 5.3 and 5.4, we compare FOVA with the most relevant schemes, namely PriVeriFL [31] and the scheme proposed by Zhang et al. [35]. They are also based on Paillier, and as shown in Table 1, they are the only two schemes that are secure under MIA while provide aggregation unforgeability. We also include BatchCrypt as a baseline. To provide a comprehensive comparison, we evaluate all four schemes in two representative FL settings:

- Cross-silo setting in section 5.3: A small number of clients with sufficient computational resources operate in a reliable network. We set the number of clients to 10 and vary the vector size from 10^2 to 10^7 . Since clients in this setting are assumed to have abundant resources, experiments are conducted on a server equipped with an AMD Ryzen Threadripper 3990X CPU, an NVIDIA RTX 3090 GPU, and 128 GB RAM. As the network is assumed to be reliable, client dropout is not considered. The online acceleration and the key generation optimization for FOVA introduced in Section 4.1 are disabled in this setting.
- Cross-device setting in section 5.4: A large number of clients with limited resources participate over an unreliable network. We fix the vector size at 1000 and vary the number of clients from 10^2 to 10^3 . As clients in this setting have constrained resources, experiments are conducted on a Single Board Computer (SBC) with a Rockchip(R) RK3399 CPU (dual-core Cortex-A72 up to 1.8 GHz and quad-core Cortex-A53 up to 1.4 GHz) and 4 GB RAM. This class of SBCs is widely used in IoT, edge computing, and computer vision applications, making it representative for cross-device scenarios. Client dropout is considered in this setting due to the unreliable network. Experiments are conducted assuming each client drops out arbitrarily with a probability of 10% or 20%. Therefore, dropout robustness is achieved by the second strategy demonstrated in Section 4.2. In this setting, the online acceleration for FOVA introduced in Section 4.1 is enabled, as preprocessing plays a crucial role in resource-constrained environments.

5.2 Concrete Cost of Enhancing Paillier-based SA to FOVA

In this section, we quantify the concrete overhead introduced by enhancing conventional Paillier-based SA to FOVA. As discussed in Section 5.1, we adopt BatchCrypt as a representative Paillier-based SA scheme and conduct experiments on two representative applications.

Computational Overhead The detailed comparison of computational overhead is shown in Fig. 4. It can be observed that the enhancement introduces additional computational overhead during the encryption and verification phases. However, a reduction in computational cost is observed during the aggregation phase. We will provide a detailed explanation of this interesting reduction later.

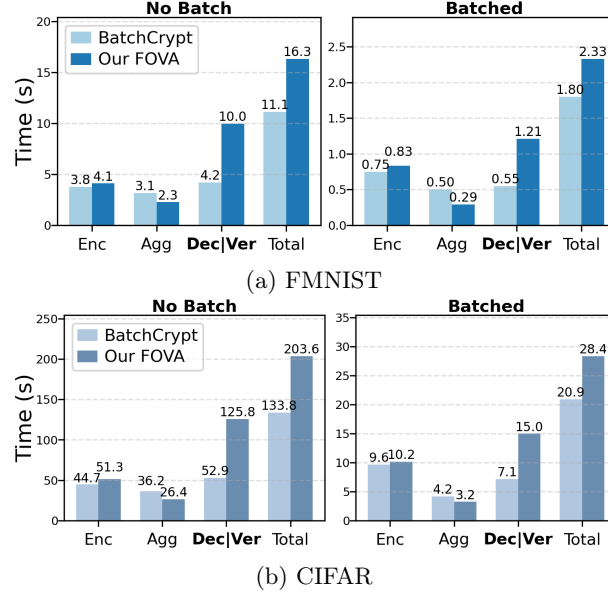


Fig. 4: Breakdown of training iteration time across different datasets and models. Note that model computation is excluded from this analysis, as the focus is solely on the overhead introduced by the authenticity mechanism.

Let us first focus on the comparison data without batch optimization, as this subset is unaffected by batching and thus directly reflects the cost introduced by the added authenticity mechanisms. As shown in Table 2, during the encryption phase, our verifiable aggregation scheme incurs a 9.04% increase in computational overhead when training on the FMNIST, and a 14.78% increase when training on the CIFAR. These increases are primarily attributed to the computation of the PRF, which is the only additional operation in FOVA compared to the conventional Paillier-based secure aggregation. In the aggregation phase, however, FOVA achieves an average 27.62% reduction in computational overhead. Although one might intuitively expect that the added authenticity would incur

a higher computational cost, FOVA actually reduces the computational burden during the aggregation phase for a straightforward reason. In the conventional Paillier-based SA scheme, the aggregation algorithm $\text{HA}(\text{param}, (X_1, \dots, X_n))$ computes $\prod_{i=1}^n X_i \bmod N^2$. In contrast, FOVA’s aggregation algorithm computes $\prod_{i=1}^n X_i \bmod kN^2$, where k is a λ -bit prime. Since kN^2 is k times larger than N^2 , the modular operation in FOVA takes less time, resulting in a reduction in computational cost during the aggregation phase. In the verification phase, the introduction of authenticity leads to an average 137.69% increase in computational cost compared to standard decryption. However, we argue that this increase is acceptable for two reasons: (1) The additional computational cost allows the Paillier-based SA scheme to be seamlessly extended to FOVA, without the need for introducing new cryptographic primitives or altering the existing framework. This is essential for ensuring compatibility between FOVA and well-established optimizations for conventional Paillier-based SA. Moreover, this extension allows the easy deployment of the conventional Paillier-based SA in malicious settings, thereby enhancing its security with minimal changes to the original system. (2) The overall increase remains modest. In fact, the total time cost increase by only $46.98 \sim 52.15\%$, which demonstrates that the additional overhead is relatively small in the context of the overall training process. (3) Although there is an average 49.57% relative increase in overall computational overhead, FOVA is still faster than the most relevant schemes. Experiment results are demonstrated in Section 5.3.

Table 2: Time comparison between FOVA and BatchCrypt without batch optimization. Since BatchCrypt does not support authenticity, this comparison reflects the cost of the enhancement.

Model	Scheme Phase	BatchCrypt	FOVA	+/-
FMNIST	Encryption	3.76 s	4.10 s	+9.04%
	Aggregation	3.15 s	2.26	-28.25%
	Decryption/ Verification	4.20 s	9.97 s	+137.38%
	Total	11.11 s	16.33 s	+46.98%
CIFAR	Encryption	44.73 s	51.34 s	+14.78%
	Aggregation	36.21 s	26.44 s	-26.98%
	Decryption/ Verification	52.86 s	125.80 s	+137.99%
	Total	133.80 s	203.58 s	+52.15%

Communication Overhead Regarding communication overhead, the data transmitted from clients to the server in FOVA is identical to that in conventional Paillier-based SA, since both consist solely of standard Paillier ciphertexts. The difference lies in the server-to-client communication, where FOVA incurs a 25% increase. This overhead arises from the larger modulus used in the aggregation

phase, which increases the bit length of the aggregated ciphertexts. Concretely, in the conventional scheme, the aggregated ciphertext sent by the server to each client has a bit length of 4λ , whereas in FOVA it increases to 5λ , i.e., a 25% size increase.

5.3 Comparison in Cross-Silo Scenario

In the following, we present a detailed comparison of the computational and communication costs of the schemes in the cross-silo setting. The comparison includes FOVA, PriVeriFL, and the scheme by Zhang et al., with BatchCrypt serving as a baseline.

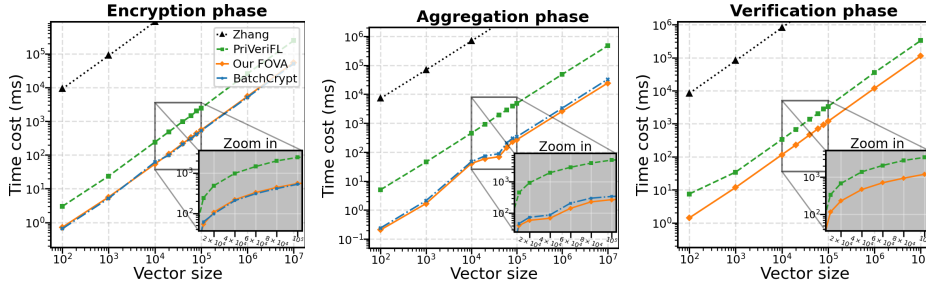


Fig. 5: Computational overhead in different phases under varying vector sizes. The secure aggregation scheme BatchCrypt is included in both encryption and aggregation phase as a baseline and is excluded in the verification phase since it does not support the verification of the aggregated result.

Computational Overhead At first glance, the comparison results presented in Fig. 5 indicate that the scheme proposed by Zhang et al. incurs significantly higher time overhead than the other three schemes across all three phases. A detailed analysis of this phenomenon is provided later. Among the remaining schemes, PriVeriFL consistently exhibits higher time overhead than both FOVA and BatchCrypt in all phases. In contrast, FOVA demonstrates time overheads in the encryption and aggregation phases that are comparable to those of BatchCrypt. A detailed breakdown and analysis of these observations are provided below.

The most salient feature in Fig. 5 is that the scheme proposed by Zhang et al. incurs time overheads that are 3 to 4 orders of magnitude higher than those of the other schemes across all phases. This overhead primarily stems from the use of bilinear pairings in their design. Specifically, bilinear pairings rely on elliptic curve groups, and group operations over elliptic curves incur significantly greater computational costs than those over multiplicative groups modulo a prime. Such overhead is unavoidable in this class of verifiable aggregation protocols that combine Paillier with bilinear pairings.

Another class of verifiable aggregation protocols, based on Paillier and homomorphic hash functions (HHF), demonstrates a substantial advantage in time

efficiency over pairing-based schemes. However, as illustrated in Fig. 5, PriVeriFL, despite avoiding bilinear pairings, still incurs approximately one order of magnitude higher computational overhead than FOVA in the encryption and aggregation phases, and about twice as much overhead in the verification phase. This additional computational overhead is expected, as PriVeriFL not only performs encryption, aggregation, and decryption using the Paillier, but also relies on digital signatures and HHF to ensure the unforgeability of the aggregated result. In contrast, FOVA achieves unforgeability and soundness solely by leveraging the inherent properties of Paillier, thereby avoiding the extra cost introduced by auxiliary cryptographic primitives.

In this setting, compared to BatchCrypt, FOVA introduces a 6 ~ 9% increase in time overhead during the encryption phase, while achieves an average 21.41% reduction in the aggregation phase. The detailed explanations are demonstrated in Section 5.2.

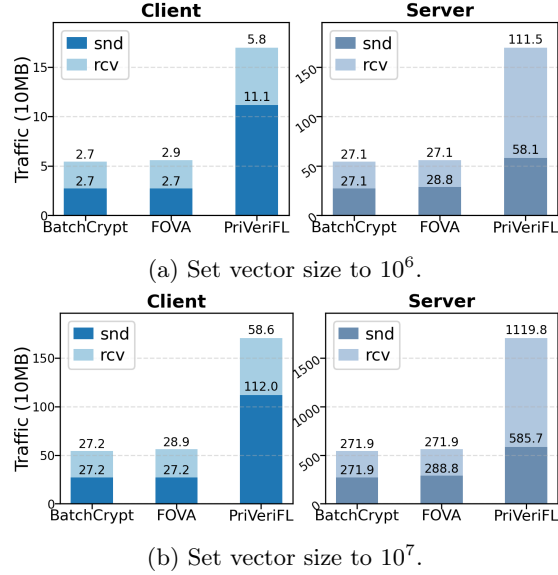


Fig. 6: Comparison of the communication overhead incurred in one training iteration under varying vector sizes. The secure aggregation scheme BatchCrypt is included as a baseline.

Communication Overhead Next, we refer to Fig. 6, where we measured the communication overhead of FOVA, BatchCrypt, and PriVeriFL at vector sizes of 10^6 and 10^7 . The scheme proposed by Zhang et al. was excluded from this comparison due to its prohibitively high time overhead. Specifically, at a vector size of 10^6 , the encryption, aggregation, and decryption phases of Zhang et al.’s scheme take approximately 23 hours, 19 hours, and 22 hours, respectively. This highlights the impracticality of their solution for real-world deployment. Given

this limitation, we exclude Zhang et al.’s scheme from further communication analysis and instead focus on comparing the communication overhead of FOVA and PriVeriFL, using BatchCrypt as the baseline.

At first glance, the communication overhead of FOVA is comparable to that of BatchCrypt and significantly lower than that of PriVeriFL. On the client side, FOVA and BatchCrypt transmit the same amount of data, as both involve sending Paillier ciphertexts. However, the amount of data received by the client in FOVA is larger than that in BatchCrypt. This is because the modulus used in the ciphertexts returned by the server is larger in FOVA, resulting in longer bit-lengths of the aggregated ciphertexts. This difference is also reflected in the amount of data sent from the server. As for PriVeriFL, its client-side transmission volume is 4.1 times larger than that of FOVA. This is because, in addition to sending Paillier ciphertexts, PriVeriFL requires the client to transmit HHF values and their signatures, the size of the training dataset along with its signature, as well as ciphertexts resulting from the symmetric encryption of the split gradient parameters. On the server side, PriVeriFL requires the server to send each client the HHF values and their corresponding signatures from all clients, as well as the sizes of their training datasets and associated signatures. In addition, the server must also transmit the aggregated Paillier ciphertexts and related data. As a result, the amount of data the server needs to send in PriVeriFL is 2.1 times greater than in FOVA.

5.4 Comparison in Cross-Device Scenario

In the following, we present a detailed comparison of the computational and communication costs of the schemes in the cross-device setting. The comparison includes FOVA, PriVeriFL, with BatchCrypt serving as a baseline. The scheme proposed by Zhang et al. is excluded from the cross-device setting, as its high computational overhead makes it impractical for resource-constrained devices.

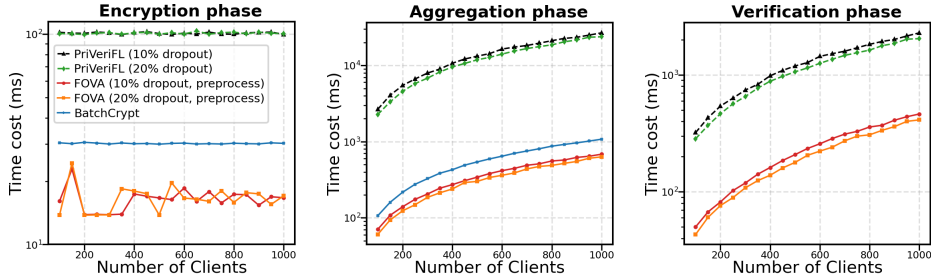


Fig. 7: Computational overhead in different phases under varying client numbers and dropout rates. The secure aggregation scheme BatchCrypt is included in both encryption and aggregation phase as a baseline and is excluded in the verification phase since it does not support the verification of the aggregated result.

Computational Overhead As shown in Fig. 7, FOVA incurs less computational overhead than PriVeriFL, and even BatchCrypt, regardless of the dropout rate. A detailed analysis of this phenomenon is provided later. Another feature in this cross-device setting is that, during the encryption phase, neither client dropout nor an increase in the number of clients affects the computational cost. This holds for all three schemes, as the encryption phase only involves computing Paillier ciphertexts, which is independent of the client number. The only exception is that PriVeriFL additionally requires signature generation, which is likewise independent of the number of clients.

In this cross-device setting, FOVA incurs lower computational overhead than both PriVeriFL and BatchCrypt during the encryption phase. The reason for the reduction compared to PriVeriFL has already been explained in Section 5.3; here we focus on the comparison with the baseline scheme BatchCrypt. In BatchCrypt, the encryption phase requires generating random values and computing their N th power modulo N^2 , where N is the product of two λ -bit primes. In FOVA, by contrast, preprocessing is introduced. Specifically, FOVA generates pseudorandom values via the PRF and also computes their N th power modulo N^2 . Since the inputs to the PRF are already known to the clients during the setup phase, both the invocation of PRF and the modular exponentiation can be performed in advance, before the encryption phase. While preprocessing could also be applied to BatchCrypt by generating random values and performing the same computation ahead of time, we intentionally treat the two schemes differently to highlight the effectiveness of preprocessing in FOVA.

During the aggregation phase, FOVA continues to incur lower computational overhead than both PriVeriFL and BatchCrypt in the cross-device setting, consistent with the results in the cross-silo setting. Compared to BatchCrypt, FOVA achieves an average reduction of 36.00% and 43.07% when clients drop out with probabilities of 10% and 20%, respectively. This reduction stems from the same reason as in the cross-silo setting, while the decrease in the number of online clients further contributes to the improvement. PriVeriFL, by contrast, incurs two orders of magnitude more time overhead than FOVA in the cross-device setting. The underlying reason is the same as in the cross-silo setting, however, in the cross-silo case, PriVeriFL’s overhead exceeds FOVA’s by only one order of magnitude. The limited computing resources in the cross-device scenario amplify this performance gap, highlighting the adaptability of FOVA to resource-constrained environments. A similar phenomenon occurs in the verification phase: PriVeriFL incurs roughly one order of magnitude more time overhead than FOVA in the cross-device setting, whereas in the cross-silo scenario the factor is only about two.

Communication Overhead In Table 3, we compare the communication overhead of FOVA and PriVeriFL. On the client side, the traffic of both schemes remains below 1 MB. Client dropout does not affect the amount of data sent by each client, but it does impact the amount of data received. Specifically, in FOVA, each client receives the aggregated result together with the IDs of offline clients

from the server. Hence, as the number of dropouts increases, the server must send more IDs to each online client, leading to higher receiving overhead. In contrast, PriVeriFL requires the server to distribute every online client’s signature to all other online clients. As a result, when the number of dropouts increases, the total data received by each online client decreases. This observation also explains the traffic pattern on the server side: as the dropout probability increases, the server in FOVA must transmit more data, whereas in PriVeriFL the server transmits less.

Another observation is that the growth in server-side communication overhead with respect to the number of clients is significantly smaller in FOVA than in PriVeriFL. As shown in Table 3, when the number of clients increases from 500 to 1000, the server-side communication overhead in FOVA grows by 143% and 115% under the two dropout probabilities, whereas in PriVeriFL it grows by 225% and 241%, respectively. This result demonstrates that FOVA scales more efficiently than PriVeriFL, highlighting its adaptability in cross-device scenarios.

Table 3: Communication overhead incurred in one training iteration.

(a) Set number of clients to 500.

Traffic (MB) \ Scheme		10% Dropout		20% Dropout	
		FOVA	PriVeriFL	FOVA	PriVeriFL
Client	Send	0.03	0.11	0.03	0.11
	Receive	0.03	0.17	0.04	0.16
Server	Send	15.00	77.76	16.29	63.84
	Receive	12.46	51.45	11.07	44.47

(b) Set number of clients to 1000.

Traffic (MB) \ Scheme		10% Dropout		20% Dropout	
		FOVA	PriVeriFL	FOVA	PriVeriFL
Client	Send	0.03	0.11	0.03	0.11
	Receive	0.04	0.28	0.04	0.26
Server	Send	36.52	253.00	35.07	217.72
	Receive	24.69	101.78	22.25	92.00

6 Conclusion and Future Work

In this paper, we present FOVA, a secure aggregation protocol that simultaneously achieves aggregation hiding and authenticity while supporting one-shot clients. FOVA is built upon a verifiable linearly homomorphic encryption scheme, for which we provide a new construction based solely on the Paillier cryptosystem. This design ensures high efficiency and allows FOVA to be integrated into existing Paillier-based FL frameworks with minimal modifications. We implement and

evaluate FOVA in representative scenarios, demonstrating that it is practical for real-world deployment. In addition, we open-source FOVA together with a plugin that upgrades existing Paillier-based secure aggregation protocols to FOVA. An important direction for future work is to remove the trusted dealer assumption, enabling the protocol to be deployed in a broader range of settings.

A FOVA with Input Validation

In this section, we show how to enhance FOVA with input validation to defend against poisoning attacks. As discussed in Section 1, input validation is typically achieved by proving in zero-knowledge that the L_2 or L_∞ norm of the input vector is bounded under encryption. This requires an NIZK range proof for Paillier ciphertexts, with concrete instances given in [8,18,19,4]. However, in FOVA, the Paillier encryption key is hidden from the server, while in NIZKs the verifier must know the key. Therefore, the server cannot serve as verifier. Instead, we extend the system model by adding a verifier role in addition to clients and the server. The verifier is assumed to be semi-honest and will not collude with the clients or the server. The verifier could be external auditors, the dealer or the system administrators.

Suppose each client P_i holds a vector $\mathbf{x}_i \in \mathbb{Z}_N^{k5}$, where k denotes the vector dimension. Each client is required to prove that the L_∞ norm of $\mathbf{x}_i$ is bounded by δ . The challenge is that the client must convince the verifier of this bound without revealing $\mathbf{x}_i$. Concretely, client P_i encrypts each element of $\mathbf{x}_i$ to obtain the ciphertext vector $\mathbf{X}_i$, and then proves in zero-knowledge that the plaintext vector underlying $\mathbf{X}_i$ satisfies $\|\mathbf{x}_i\|_\infty < \delta$. In FOVA, it is actually required to prove in zero-knowledge that the plaintext underlying each Paillier ciphertext is in the range $[0, \delta]$. Therefore, enhancing FOVA to support input validation ultimately reduces to constructing a zero-knowledge proof for the following relation R , for which efficient instantiations have already been provided in prior work [8,18,19,4].

$$R = \left\{ (X, N), (x, r) : X = (1 + N)^x \cdot r^N \bmod N^2 \wedge x \in \mathbb{Z}_\delta \right\}.$$

A.1 Modifications in FOVA

In the following, we show how to modify FOVA to adopt the zero-knowledge proof system. First, except with the clients and the server, a verifier is added to the protocol. It is used to verify the L_∞ norm of input from the clients. As discussed before, it is assumed to be semi-honest and not colluding with the clients or the server. We assume that the concrete bound of the vectors δ is known to all the parties. The details are given below.

Setup phase. The setup phase is the same as that in FOVA, except that the one-time dealer or the key distribution service computes $(\text{param}, \text{sk}) \leftarrow \text{Gen}(1^\lambda, s)$, where $\text{sk} = (N, \Phi(N))$, and securely sends the parameter N to the verifier.

⁵ In certain scenarios (e.g., federated learning), the input vectors may contain elements from $\mathbb{R}$. In such cases, an encoding mechanism is required to map these real-valued elements into $\mathbb{N}$

Encryption phase. In FOVA, each client P_i encrypts each privacy input using the secret key sk and sends the ciphertext X_i to the server. To support input validation, it is required to compute the proof π_i of the relation R , and send (i, X_i, π_i) to the verifier while send (i, X_i) to the server.

Aggregation phase. In the aggregation phase, the server performs the aggregation after receiving the certification from the verifier. Specifically, the verifier receives (i, X_i, π_i) from each online client. After that, it verifies the proof of each online client and computes $\text{Cert}_i := H(X_i)$ if success. Let $\mathbf{I}_{\text{verify}}$ denote the ID set of those clients whose proof passes the verification. The verifier then sends (i, Cert_i) , where $i \in \mathbf{I}_{\text{verify}}$, to the server. The server receives (i, X_i) from client $P_i (i \in \mathbf{I}_{\text{online}})$ and $(i', \text{Cert}_{i'})$ for $i' \in \mathbf{I}_{\text{verify}}$ from the verifier. Then, for each $i \in \mathbf{I}_{\text{verify}}$, the server checks whether $H(X_i)$ equals to Cert_i . Let $\mathbf{I}_{\text{agg}}$ denote the set of client IDs who pass the check. The server performs the aggregation algorithm on the X_i , where $i \in \mathbf{I}_{\text{agg}}$, to obtain Y . At last, the server sends $\mathbf{I}_{\text{agg}}$ to each client $P_i (i \in \mathbf{I}_{\text{agg}})$.

Verification phase. The verification phase is the same as that in FOVA.

B Security Proofs

B.1 Proof of Theorem 1

Proof. The correctness of Π^+ follows directly from the standard Paillier. We refer to [14] for the details. Here, we focus on proving the completeness.

Following the completeness definition in Definition 1, we have

$$Y = \prod_{i=1}^n X_i \bmod \text{pp} = (1 + N)^{\sum_{i=1}^n x_i} \cdot \left(\prod_{i=1}^n r_i \right)^N \bmod N^2.$$

This is exactly the value of Y' computed in the first step of the verification algorithm `Verify`. Therefore, `Verify` outputs 1 in its final step, completing the proof of completeness. $\square$

B.2 Proof of Theorem 2

Proof. Since the encryption algorithm in Π^+ is identical to that in Paillier and no additional information leak, the privacy of Π^+ follows directly from the IND-CPA security of standard Paillier. We refer to [14] for the details. Here, we focus on proving the authenticity.

To prove the authenticity, let's start with the process of the verification algorithm `Verify` in Construction 1. On input the secret key sk , the decryption result y , n random values $(r_1, \dots, r_n)$ used in encryption process and an original ciphertext Y , the verification algorithm computes a test ciphertext $Y' := (1 + N)^y \cdot \left(\prod_{i=1}^n r_i \right)^N \bmod N^2$. If $(Y \bmod N^2) \neq Y'$, the algorithm returns $\perp$. First, if a malicious adversary $\mathcal{A}$ attempts to forge Y by omitting any ciphertexts, the verification algorithm `Verify` will immediately detect this, since it knows all n random values embedded in the ciphertexts. In this case, `Verify` will find that

$(Y \bmod N^2) \neq Y'$ and output $\perp$. Therefore, to prove authenticity, it suffices to prove that, given the ciphertexts, any PPT adversary can forge a valid ciphertext Y only with negligible probability. Formally, for any Y output by the adversary $\mathcal{A}$ defined in the $\text{Adv}_{\mathcal{A}, \Pi}^{\text{Aut}}$ game, the following condition holds during the verification process:

$$\Pr \left[\begin{array}{c} y \neq \sum_{i=1}^n x_i \\ \cap \\ (Y \bmod N^2) = Y' \end{array} \right] \leq \text{negl}(\lambda). \quad (2)$$

In the following, we analyze the concrete probability defined in (2) from the perspective of the adversary $\mathcal{A}$, whose goal is to forge a value Y that satisfies the equation $(Y \bmod N^2) = Y'$ while ensuring that $y \neq \sum_{i=1}^n x_i$. Intuitively, the negligible probability result follows from three aspects: (1) Given the ciphertexts X_i , an adversary may arbitrarily multiply or add values to the true homomorphic result $T := \prod_{i=1}^n X_i$, but the only two valid forms of forgery are $Y = T \cdot (1 + aN)$ or $Y = T + aN$, where $a \xleftarrow{\$} \mathbb{Z}_N$. (2) The adversary only knows kN^2 , from which recovering N is possible only with negligible probability. (3) Without knowing N , the adversary can only guess the values $(1 + aN)$ or aN within $\mathbb{Z}_{kN^2}$, succeeding with negligible probability. Specifically, given $X_1, \dots, X_n$, the adversary can obtain

$$T := \prod_{i=1}^n X_i \equiv (1 + N)^{\sum_{i=1}^n x_i} \cdot \left(\prod_{i=1}^n r_i \right)^N \pmod{N^2}.$$

Note that forging a valid Y can only be achieved through one of the following two types of malicious strategies, or a combination thereof:

- Set $Y := T \cdot (1 + aN)$, where $a \in \mathbb{Z}_N$ and $a \neq 0$. It can be easily verified that the decryption value y of the ciphertext $(Y \bmod N^2)$ is equal to $\sum_{i=1}^n x_i + a$. Therefore, we have that $y \neq \sum_{i=1}^n x_i$, while the equation $(Y \bmod N^2) = Y'$ still holds.
- Set $Y := T + aN$, where $a \in \mathbb{Z}_N$ and $a \neq 0$. First, executing the decryption algorithm on the ciphertext $(Y \bmod N^2)$ gives us

$$\begin{aligned} y &:= \text{Dec}(\text{dk}, Y \bmod N^2) \\ &= \frac{(Y^{\Phi(N)} \bmod N^2) - 1}{N} \cdot \Phi(N)^{-1} \bmod N \\ &= \sum_{i=1}^n x_i + a \left(\prod_{i=1}^n r_i \right)^{N(\Phi(N)-1)} \bmod N. \end{aligned}$$

Since $a \neq 0$, we have that $y \neq \sum_{i=1}^n x_i$. Next, let R denote $\prod_{i=1}^n r_i$ and we can check that

$$\begin{aligned}
Y' &= (1+N)^y \cdot R^N \bmod N^2 \\
&= (1+N)^{\sum_{i=1}^n x_i} R^N \cdot (1+N)^{aR^{N(\Phi(N)-1)}} \bmod N^2 \\
&= (1+N)^{\sum_{i=1}^n x_i} R^N + a(1+N)^{\sum_{i=1}^n x_i} R^{N\Phi(N)} N \bmod N^2 \\
&= T + a(1 + \sum_{i=1}^n x_i \cdot N)N \bmod N^2 \\
&= T + aN \bmod N^2 = Y \bmod N^2
\end{aligned}$$

Therefore, we have that $y \neq \sum_{i=1}^n x_i$, while the equation $(Y \bmod N^2) = Y'$ still holds.

As a conclusion, to forge a valid Y that satisfies the equation $(Y \bmod N^2) = Y'$ while ensuring that $y \neq \sum_{i=1}^n x_i$, the adversary has to obtain $1 + aN$ or aN for any $a \neq 0 \in \mathbb{Z}_N$. However, the core of the authenticity guarantee lies in the fact that the adversary is not given N , but only the parameter $\mathbf{pp} = kN^2$, where k is a λ -bit prime, and the ciphertexts $X_1, \dots, X_n$. Due to the hardness of the factoring problem, it is computationally infeasible to derive N from $\mathbf{pp}$. Moreover, by the privacy of the verifiable linearly homomorphic encryption scheme, the ciphertexts $X_1, \dots, X_n$ are computationally indistinguishable from random elements in the ciphertext space. Therefore, we have that

$$\begin{aligned}
\text{Adv}_{\mathcal{A}, H}^{\text{Aut}}(\lambda) &= \Pr \left[\begin{array}{c} Y' = (Y \bmod N^2) \\ \cap \\ y \neq \sum_{i=1}^n x_i \end{array} \right] \\
&\approx_c \Pr \left[\begin{array}{c} A \bmod N = 0 \\ \cup \\ (A-1) \bmod N = 0 \end{array} \middle| \begin{array}{c} X_1, \dots, X_n \xleftarrow{\$} \mathbb{Z}_{N^2}^*; \\ A \leftarrow \mathcal{A}(\mathbf{pp}, X_1, \dots, X_n); \\ A \leftarrow A \bmod N^2; \end{array} \right] \\
&= \max \left\{ \frac{2(N-1)}{N^2}, \frac{1}{2^\lambda} \right\} = \text{negl}(\lambda).
\end{aligned}$$

This concludes the proof.

B.3 Proof of Theorem 3

Proof. We need to prove the three properties required for the security definition of verifiable aggregation, as defined in $\mathcal{F}^{\text{va}}$.

Input privacy: We begin by constructing a simulator Sim for $\mathcal{F}^{\text{va}}$ that produces a transcript indistinguishable from the real protocol. The simulator works as follows:

1. Sim samples a random value $s \leftarrow \{0, 1\}^\lambda$ and computes $(\mathbf{pp}, \mathbf{sk}) \leftarrow \text{Gen}(1^\lambda, s)$. Upon receiving (ssid, P_i) for all $i \in [n]$ from $\mathcal{F}^{\text{va}}$, Sim sends n and $\mathbf{pp}$ to the server (which is the adversary $\mathcal{A}$).
2. For each $i \in [n]$, Sim samples a random value $r_i \xleftarrow{\$} \mathbb{Z}_N^*$, where N is contained in $\mathbf{sk}$. After computing $X_i \leftarrow \text{Enc}(\mathbf{sk}, 0, r_i)$, Sim sends X_i to the adversary $\mathcal{A}$.

Now we show that the transcripts in the simulated world and the real world are indistinguishable. We give a sequence of games in the following, going from the real one to the simulated one.

- **Game₀**: Let this be the real world. Specifically, the transcript includes:
 1. The number of clients n and public parameter $\mathbf{pp}$ generated by the clients.
 2. The ciphertexts X_i sent by the clients, computed as
 - (a) Compute $r_i \leftarrow \text{PRF}_s(i||j)$, where s is the uniformly sampled seed and j is the iteration identifier.
 - (b) Compute $X_i \leftarrow \text{Enc}(\mathbf{sk}, x_i, r_i)$.
 - (c) Send X_i to the server (which is the adversary $\mathcal{A}$).
- **Game₁**: This game is the same as the previous one except that in step (a) r_i is uniformly sampled from $\mathbb{Z}_N^*$ instead of being generated by PRF. This game is indistinguishable from **Game₀** following the security of PRF.
- **Game₂**: This game is the same as the previous one except that in step (b) X_i is computed by $\text{Enc}(\mathbf{sk}, 0, r_i)$ instead of $\text{Enc}(\mathbf{sk}, x_i, r_i)$. This game is indistinguishable from **Game₁** following the privacy of the underlying VLHE scheme, as defined in Definition 1.

Now **Game₂** is the same as the simulated world, and therefore we have shown that through a sequence of games the two worlds are indistinguishable.

Aggregation hiding: Intuitively, aggregation hiding follows from the fact that, after executing the evaluation algorithm, the server only obtains a VLHE ciphertext. Therefore, aggregation hiding is implied by the privacy of VLHE.

Aggregation authenticity: Intuitively, the aggregate authenticity of FOVA reduces to the authenticity of the VLHE scheme Π^+ defined in Definition 1. To prove authenticity of FOVA under the malicious server, it is actually required to prove that, for any $n, x_1, \dots, x_n \in \mathbb{N}$ such that $\sum_{i=1}^n x_i \in \mathbb{Z}_N$, it holds that for any PPT adversary $\mathcal{A}$, the following advantage $\text{Adv}_{\mathcal{A}}^{\text{Unfg}}$ is a negligible function in λ .

$$\text{Adv}_{\mathcal{A}}^{\text{Unfg}}(\lambda) := \Pr \left[\begin{array}{c} y \neq \sum_{i=1}^n x_i \\ \cap \\ v \neq \perp \end{array} \middle| \begin{array}{l} (\mathbf{pp}, \mathbf{sk}) \leftarrow \text{Gen}(1^\lambda) \\ s \xleftarrow{\$} \{0, 1\}^\lambda \\ r_i \leftarrow \text{PRF}_s(i||0), i \in [n] \\ X_i \leftarrow \text{Enc}(\mathbf{sk}, x_i, r_i), i \in [n] \\ Y \leftarrow \mathcal{A}^{\text{Enc}(\mathbf{sk}, \cdot, \cdot)}(\mathbf{pp}, (X_1, \dots, X_n)); \\ y \leftarrow \text{Dec}(\mathbf{sk}, Y); \\ v \leftarrow \text{Verify}(\mathbf{sk}, y, (r_1, \dots, r_n), Y) \end{array} \right]$$

Since the PRF seed s is uniformly sampled and confidential to the adversary, we can have that r_i is computationally indistinguishable to a random value by the security of PRF. Therefore, we can remove the sampling of s and replace r_i with uniformly sampled value in $\mathbb{Z}_N^*$. This gives us the same probability definition as $\text{Adv}_{\mathcal{A}, \Pi}^{\text{Aut}}$ defined in Definition 1. Since we have proof that $\text{Adv}_{\mathcal{A}, \Pi}^{\text{Aut}} = \text{negl}(\lambda)$, we can conclude that $\text{Adv}_{\mathcal{A}}^{\text{Unfg}}(\lambda) = \text{negl}(\lambda)$. This concludes the proof. $\square$

References

1. Diego F. Aranha, Anamaria Costache, Antonio Guimarães, and Eduardo Soria-Vazquez. HELIOPOLIS: Verifiable computation over homomorphically encrypted data from interactive oracle proofs is practical. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part V*, volume 15488 of *LNCS*, pages 302–334. Springer, Singapore, December 2024.
2. James Bell, Adrià Gascón, Tancrède Lepoint, Baiyu Li, Sarah Meiklejohn, Mariana Raykova, and Cathie Yun. ACORN: Input validation for secure aggregation. In Joseph A. Calandrino and Carmela Troncoso, editors, *USENIX Security 2023*, pages 4805–4822. USENIX Association, August 2023.
3. James Bell-Clark, Adrià Gascón, Baiyu Li, Mariana Raykova, and Phillipp Schoppmann. Willow: Secure aggregation with one-shot clients. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part VIII*, volume 16007 of *LNCS*, pages 285–318. Springer, Cham, August 2025.
4. Fabrice Boudot. Efficient proofs that a committed number lies in an interval. In Bart Preneel, editor, *EUROCRYPT 2000*, volume 1807 of *LNCS*, pages 431–444. Springer, Berlin, Heidelberg, May 2000.
5. Ignacio Cascudo, Anamaria Costache, Daniele Cozzo, Dario Fiore, Antonio Guimarães, and Eduardo Soria-Vazquez. Verifiable computation for approximate homomorphic encryption schemes. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part VII*, volume 16006 of *LNCS*, pages 643–677. Springer, Cham, August 2025.
6. Sylvain Chatel, Christian Knabenhans, Apostolos Pyrgelis, Carmela Troncoso, and Jean-Pierre Hubaux. VERITAS: Plaintext encoders for practical verifiable homomorphic encryption. In Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie, editors, *ACM CCS 2024*, pages 2520–2534. ACM Press, October 2024.
7. Amrita Roy Chowdhury, Chuan Guo, Somesh Jha, and Laurens van der Maaten. EIFFeL: Ensuring integrity for federated learning. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022*, pages 2535–2549. ACM Press, November 2022.
8. Borui Gong, Wang Fat Lau, Man Ho Au, Rupeng Yang, Haiyang Xue, and Lichun Li. Efficient zero-knowledge arguments for paillier cryptosystem. In *2024 IEEE Symposium on Security and Privacy*, pages 1813–1831. IEEE Computer Society Press, May 2024.
9. S. Dov Gordon, Jonathan Katz, Feng-Hao Liu, Elaine Shi, and Hong-Sheng Zhou. Multi-client verifiable computation with stronger security guarantees. In Yevgeniy Dodis and Jesper Buus Nielsen, editors, *TCC 2015, Part II*, volume 9015 of *LNCS*, pages 144–168. Springer, Berlin, Heidelberg, March 2015.
10. Changhee Hahn, Hodong Kim, Minjae Kim, and Junbeom Hur. Versa: Verifiable secure aggregation for cross-device federated learning. *IEEE Transactions on Dependable and Secure Computing*, 20(1):36–52, 2023.
11. Shai Halevi, Yehuda Lindell, and Benny Pinkas. Secure computation on the web: Computing without simultaneous interaction. In Phillip Rogaway, editor, *CRYPTO 2011*, volume 6841 of *LNCS*, pages 132–150. Springer, Berlin, Heidelberg, August 2011.
12. Bargav Jayaraman and David Evans. Evaluating differentially private machine learning in practice. In Nadia Heninger and Patrick Traynor, editors, *USENIX Security 2019*, pages 1895–1912. USENIX Association, August 2019.

13. Harish Karthikeyan and Antigoni Polychroniadou. sfOPA: One-shot private aggregation with single client interaction and its applications to federated learning. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part VIII*, volume 16007 of *LNCS*, pages 319–353. Springer, Cham, August 2025.
14. Jonathan Katz and Yehuda Lindell. *Introduction to modern cryptography: principles and protocols*. Chapman and hall/CRC, 2007.
15. Jakub Konečný, H Brendan McMahan, Daniel Ramage, and Peter Richtárik. Federated optimization: Distributed machine learning for on-device intelligence. *arXiv preprint arXiv:1610.02527*, 2016.
16. Alex Krizhevsky. Learning multiple layers of features from tiny images. 2009.
17. Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. In F. Pereira, C.J. Burges, L. Bottou, and K.Q. Weinberger, editors, *Advances in Neural Information Processing Systems*, volume 25. Curran Associates, Inc., 2012.
18. Yehuda Lindell. Fast secure two-party ECDSA signing. In Jonathan Katz and Hovav Shacham, editors, *CRYPTO 2017, Part II*, volume 10402 of *LNCS*, pages 613–644. Springer, Cham, August 2017.
19. Yehuda Lindell and Ariel Nof. Fast secure multiparty ECDSA with practical distributed key generation and applications to cryptocurrency custody. In David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang, editors, *ACM CCS 2018*, pages 1837–1854. ACM Press, October 2018.
20. Yang Liu, Tao Fan, Tianjian Chen, Qian Xu, and Qiang Yang. Fate: An industrial grade platform for collaborative learning with data protection. *Journal of Machine Learning Research*, 22(226):1–6, 2021.
21. Varun Madathil, Arthur Lazzaretti, Zeyu Liu, and Charalampos Papamanthou. TACITA: Threshold aggregation without client interaction. *Cryptology ePrint Archive*, Paper 2025/1579, 2025.
22. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Artificial intelligence and statistics*, pages 1273–1282. PMLR, 2017.
23. Luca Melis, Congzheng Song, Emiliano De Cristofaro, and Vitaly Shmatikov. Exploiting unintended feature leakage in collaborative learning. In *2019 IEEE symposium on security and privacy (SP)*, pages 691–706. IEEE, 2019.
24. Milad Nasr, Reza Shokri, and Amir Houmansadr. Comprehensive privacy analysis of deep learning: Passive and active white-box inference attacks against centralized and federated learning. In *2019 IEEE symposium on security and privacy (SP)*, pages 739–753. IEEE, 2019.
25. Pascal Paillier. Public-key cryptosystems based on composite degree residuosity classes. In Jacques Stern, editor, *EUROCRYPT’99*, volume 1592 of *LNCS*, pages 223–238. Springer, Berlin, Heidelberg, May 1999.
26. Marco Palazzo, Florine W. Dekker, Alessandro Brighente, Mauro Conti, and Zekeriya Erkin. Privacy-preserving data aggregation with public verifiability against internal adversaries. In Davide Balzarotti and Wenyan Xu, editors, *USENIX Security 2024*. USENIX Association, August 2024.
27. Dario Pasquini, Danilo Francati, and Giuseppe Ateniese. Eluding secure aggregation in federated learning via model inconsistency. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022*, pages 2429–2443. ACM Press, November 2022.
28. Virat Shejwalkar, Amir Houmansadr, Peter Kairouz, and Daniel Ramage. Back to the drawing board: A critical evaluation of poisoning attacks on production

- federated learning. In *2022 IEEE Symposium on Security and Privacy*, pages 1354–1371. IEEE Computer Society Press, May 2022.
29. Shanghao Shi, Ning Wang, Yang Xiao, Chaoyu Zhang, Yi Shi, Y. Thomas Hou, and Wenjing Lou. Scale-MIA: A scalable model inversion attack against secure federated learning via latent space reconstruction. In *NDSS 2025*. The Internet Society, February 2025.
 30. Liwei Song and Prateek Mittal. Systematic evaluation of privacy risks of machine learning models. In Michael Bailey and Rachel Greenstadt, editors, *USENIX Security 2021*, pages 2615–2632. USENIX Association, August 2021.
 31. Lulu Wang, Mirko Polato, Alessandro Brighente, Mauro Conti, Lei Zhang, and Lin Xu. Priverifl: Privacy-preserving and aggregation-verifiable federated learning. *IEEE Transactions on Services Computing*, 18(2):998–1011, 2025.
 32. Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms, 2017.
 33. Chengliang Zhang, Suyi Li, Junzhe Xia, Wei Wang, Feng Yan, and Yang Liu. BatchCrypt: Efficient homomorphic encryption for Cross-Silo federated learning. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, pages 493–506. USENIX Association, July 2020.
 34. Jiale Zhang, Yue Liu, Di Wu, Shuai Lou, Bing Chen, and Shui Yu. Vpfl: A verifiable privacy-preserving federated learning scheme for edge computing systems. *Digital Communications and Networks*, 9(4):981–989, 2023.
 35. Xianglong Zhang, Anmin Fu, Huaqun Wang, Chunyi Zhou, and Zhenzhu Chen. A privacy-preserving and verifiable federated learning scheme. In *ICC 2020 - 2020 IEEE International Conference on Communications (ICC)*, pages 1–6, 2020.
 36. Joshua C. Zhao, Atul Sharma, Ahmed Roushdy Elkordy, Yahya H. Ezzeldin, Salman Avestimehr, and Saurabh Bagchi. Loki: Large-scale data reconstruction attack against federated learning through model manipulation. In *2024 IEEE Symposium on Security and Privacy*, pages 1287–1305. IEEE Computer Society Press, May 2024.